

AP COMPUTER SCIENCE A · THE UNIVERSAL STUDY GUIDE

Java + OOP — targeting a 3, 4, or 5

Tier badges: [3] foundational · [4] adds for 4+ · [5] adds for 5

Targeting a 4 → study [3] AND [4] items. Targeting a 5 → study all three tiers.

PAGE 1 | MATH OF 3/4/5 · 10 UNITS · JAVA SYNTAX HEAD-START

About this exam. AP CS A is a **Java exam** — full Java syntax, real OOP, classes/methods/inheritance. Different from AP CSP (which is broader + uses pseudocode). Half the test is MCQ trace-the-code, half is FRQ where you **WRITE** actual Java code by hand. **Indentation, semicolons, and braces matter.**

THE MATH OF YOUR TARGET

Target	MCQ (40q, 50%)	FRQ (4q, 50%)	Approx %	Tier
3 (PASS)	~22 / 40 (~55%)	~16/36 raw	~46-50%	[3] only
4	~28 / 40 (~70%)	~22/36 raw	~62-67%	[3] + [4]
5	~33 / 40 (~83%)	~28-32/36 raw	~78-82%	All

Time: 90 min for 40 MCQ (~2:15 each), 90 min for 4 FRQ (~22 min each). The 4 FRQs are: **(Q1) methods + control flow**, **(Q2) class design**, **(Q3) array/ArrayList**, **(Q4) 2D arrays**.

THE 10 UNITS — by exam weight

#	Unit	Weight	Focus
1	Primitive Types	2.5–5%	int, double, boolean, casting
2	Using Objects	5–7.5%	String, Math class, wrapper classes (Integer, Double)
3	Boolean Expressions & if	15–17.5%	Conditionals, short-circuit, De Morgan's
4	Iteration	17.5–22.5%	while, for, nested loops, String traversal
5	Writing Classes	5–7.5%	Constructors, instance vars, methods, encapsulation, toString
6	Array	10–15%	1-D arrays, traversal, common algorithms
7	ArrayList	2.5–7.5%	ArrayList<E> methods, traversal, remove pitfall
8	2D Array	7.5–10%	Row-major traversal, nested loops
9	Inheritance	5–10%	extends, super, polymorphism, Object, equals
10	Recursion	5–7.5%	Base case + recursive call, common patterns

Units 3, 4, 6, 8 = ~50% of the exam combined. Iteration + arrays + 2D arrays dominate — expect heavy MCQ tracing.

JAVA HEADER YOU'LL SEE [3]

```
public class MyClass {
    private int x;                // INSTANCE variable
    private static final int MAX = 100; // CLASS CONSTANT

    public MyClass(int initialX) { // CONSTRUCTOR
        x = initialX;
    }

    public int getX() { return x; } // ACCESSOR (getter)
    public void setX(int newX) { x = newX; } // MUTATOR (setter)
}
```

KEY INSIGHT [3] AP CS A FRQs let you **USE** methods that have been described — even if not implemented. If the question describes a method `getName()` returning a `String`, you can call it without seeing its code. Read the class header carefully.

PAGE 2 | UNITS 1-2: PRIMITIVES · STRINGS · MATH · WRAPPERS

Primitive Types [3]

```

int      whole numbers (e.g., 5, -3, 1000000)
double   decimals (e.g., 3.14, -0.5, 2.0)
boolean  true / false (lowercase!)
char     single character ('A', '7') (NOT often tested directly)

INTEGER DIVISION: 9 / 4 = 2 (NOT 2.25!)
MODULO:           9 % 4 = 1 (remainder)
CASTING:          (double) 9 / 4 = 2.25 (cast first thing)
                  (int) 9.7 = 9 (truncates, NOT rounds)

```

WATCH OUT [3] Integer division is the #1 MCQ trap. $5/2 = 2$, NOT 2.5. Cast at least one operand to double: `(double) 5 / 2 = 2.5`.

String Class [3]/[4]

```

String s = "Hello";
s.length()           // 5
s.substring(1, 4)     // "ell" (start INCLUSIVE, end EXCLUSIVE)
s.substring(2)        // "llo" (from index 2 to end)
s.indexOf("l")        // 2 (first occurrence; -1 if not found)
s.charAt(0)           // 'H'
s.equals("hello")     // false (case-sensitive)
s.equalsIgnoreCase("hello") // true
s.compareTo("Help")   // negative (s comes before alphabetically)
s + " World"          // "Hello World" (concatenation)

```

WATCH OUT [3] NEVER use `==` on Strings (or any object). `s == "Hello"` compares REFERENCES, not content. Use `s.equals("Hello")` always.

Math Class [3]

```

Math.abs(-5)         // 5
Math.pow(2, 8)        // 256.0 (returns double!)
Math.sqrt(25)         // 5.0 (returns double!)
Math.random()         // double in [0, 1)

// Random int in [low, high]:
(int)(Math.random() * (high - low + 1)) + low

```

Wrapper Classes [4]

```

Integer i = 5;        // autoboxing: int → Integer
int x = i;             // unboxing: Integer → int

// Why wrappers? ArrayList<Integer> works; ArrayList<int> does NOT.
Integer.parseInt("42") // 42
Integer.MAX_VALUE      // 2147483647

```

PAGE 3 | UNITS 3-4: CONDITIONALS · LOOPS · STRING TRAVERSAL

Boolean Operators & Short-Circuit [3]/[4]

```
&&  AND   (both must be true)
||   OR    (either must be true)
!    NOT   (flips)
```

SHORT-CIRCUIT:

```
A && B   - if A is false, B not evaluated
A || B   - if A is true, B not evaluated
```

USE for safety:

```
if (arr != null && arr.length > 0) { ... } // safe
if (i < arr.length && arr[i] > 0) { ... } // avoids out-of-bounds
```

De Morgan's Laws [4]

```
!(A && B) == !A || !B
!(A || B) == !A && !B
```

EXAMPLES:

```
!(x > 0 && y > 0) == x <= 0 || y <= 0
!(name.equals("Bob") || age > 18) == !name.equals("Bob") && age <= 18
```

for vs while [3]

```
// FOR loop (when count is known)
for (int i = 0; i < n; i++) { ... }

// WHILE loop (when condition-driven)
while (condition) { ... }

// ENHANCED FOR ("for-each") - read-only, can't modify by index
for (int x : arr) { ... }
for (String s : list) { ... }
```

WATCH OUT [3] Enhanced for loop **CANNOT** modify array elements. `for (int x : arr) { x = 0; }` does NOT change the array — x is a copy. Use index-based for loops to modify.

String Traversal — common patterns [3]/[4]

```
// Count vowels in a String
int count = 0;
for (int i = 0; i < s.length(); i++) {
    char c = s.charAt(i);
    if ("aeiouAEIOU".indexOf(c) != -1) count++;
}

// Reverse a String
String rev = "";
for (int i = s.length() - 1; i >= 0; i--) {
    rev += s.charAt(i);
}
```

Nested Loops & Off-by-One [3]/[4]

- **[3] Bounds check carefully.** `i < arr.length` (NOT `<=`). For 2D: `r < rows` and `c < cols`.
- **[3] Inner loop runs FULLY for each outer iteration.** Trace nested loops on paper.
- **[4] Pattern:** outer loop = ROWS, inner loop = COLUMNS for 2D arrays. We'll see this on Page 7.

PAGE 4 | UNIT 5: WRITING CLASSES — THE FRQ Q2 SUBJECT

FRQ Q2 is virtually always: "Write the COMPLETE class." They give a partial spec; you build the full class with constructor, instance variables, getters, and required methods. **Memorize the pattern.**

Anatomy of a Class — the universal template [3]/[4]

```
public class Book {
    // 1. INSTANCE VARIABLES (always private)
    private String title;
    private String author;
    private int pages;

    // 2. CONSTRUCTOR
    public Book(String t, String a, int p) {
        title = t;
        author = a;
        pages = p;
    }

    // 3. ACCESSORS (getters)
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public int getPages() { return pages; }

    // 4. MUTATORS (setters) — only if needed
    public void setPages(int p) { pages = p; }

    // 5. OTHER METHODS
    public boolean isLong() { return pages > 500; }

    // 6. toString (often required on FRQ)
    public String toString() {
        return title + " by " + author + " (" + pages + " pages)";
    }
}
```

Encapsulation [3]/[4]

- **Private [3]** = accessible only inside the class. Use for instance variables.
- **Public [3]** = accessible from anywhere. Use for constructors + methods other code calls.
- **[4] Why encapsulation?** Prevents external code from corrupting state. Forces use of methods → easier to maintain + validate.

static vs Instance [4]/[5]

```
STATIC = belongs to the CLASS, not any object
Math.sqrt(...) // call without making a Math object
public static int counter; // shared by all instances

INSTANCE = belongs to ONE object
Book b1 = new Book(...);
b1.getTitle(); // each Book has its own title

STATIC method CANNOT use instance variables directly.
CONSTANTS are typically: public static final int MAX = 100;
```

this Keyword [4]

```
public Book(String title, String author, int pages) {
    this.title = title; // 'this.title' = instance var; 'title' = parameter
    this.author = author;
    this.pages = pages;
}
```

[4] 'this' disambiguates when a parameter has the same name as an instance variable. Common in constructors.

PAGE 5 | UNIT 6: ARRAYS — TRAVERSAL · ALGORITHMS

Array Basics [3]

```
int[] nums = {5, 2, 8, 1, 9};           // initialized literal
int[] empty = new int[10];             // 10 elements, all 0
String[] names = new String[5];        // 5 elements, all null

nums.length      // 5 (no parentheses!)
nums[0]          // 5 (first element, ZERO-INDEXED)
nums[nums.length - 1] // 9 (last element)

// Standard traversal
for (int i = 0; i < nums.length; i++) {
    System.out.println(nums[i]);
}
```

Array Algorithms — the patterns [3]/[4]

```
// FIND MAX
int max = arr[0];
for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) max = arr[i];
}

// SUM / AVERAGE
int sum = 0;
for (int x : arr) sum += x;
double avg = (double) sum / arr.length;

// COUNT how many match a condition
int count = 0;
for (int x : arr) {
    if (x > 50) count++;
}

// SHIFT elements left by 1
for (int i = 0; i < arr.length - 1; i++) {
    arr[i] = arr[i + 1];
}
```

Linear Search & Bubble/Selection/Insertion [4]/[5]

```
// LINEAR SEARCH: worst case n comparisons
int idx = -1;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == target) { idx = i; break; }
}

// BUBBLE SORT (high-level):  $O(n^2)$ 
//   compare adjacent pairs, swap if out of order, repeat n times

// SELECTION SORT: find min in unsorted portion, swap to front
// INSERTION SORT: take each new element, insert into sorted prefix

// BINARY SEARCH (REQUIRES SORTED!):  $O(\log n)$ 
//   check middle; recurse on left or right half
```

KEY INSIGHT [4]/[5] You don't have to MEMORIZE bubble/selection/insertion sort verbatim — but you DO need to TRACE them on MCQs.
Common question: "After 2 passes of selection sort, what does arr look like?"

PAGE 6 | UNIT 7: ARRAYLIST — DYNAMIC LISTS · THE REMOVE PITFALL

ArrayList Basics [3]/[4]

```
import java.util.ArrayList;

ArrayList<String> list = new ArrayList<String>();
list.add("apple");           // append
list.add(0, "banana");       // insert at index 0
list.size()                  // 2
list.get(0)                   // "banana"
list.set(1, "cherry")         // replace; returns old value
list.remove(0)                // removes index 0; returns removed
list.contains("cherry")       // true
list.indexOf("cherry")        // index or -1
list.isEmpty()                // false
```

WATCH OUT [3] Array vs ArrayList syntax differs: `arr.length` (no parens, primitive arrays). `list.size()` (parens, ArrayList). MCQs love testing this distinction.

THE REMOVE PITFALL — must memorize [3]/[4]

```
// BUG: removing while iterating with i++
for (int i = 0; i < list.size(); i++) {
    if (list.get(i).equals("x"))
        list.remove(i);           // skips next element!
}

// FIX: count DOWN, or DON'T increment after removing
for (int i = list.size() - 1; i >= 0; i--) {
    if (list.get(i).equals("x"))
        list.remove(i);
}

// OR:
int i = 0;
while (i < list.size()) {
    if (list.get(i).equals("x")) list.remove(i);
    else i++;
}
```

KEY INSIGHT [4] Why does removing skip? When you remove index 3, item at index 4 shifts to 3. Then `i++` makes you check index 4 (which was originally 5) — index 3 (originally 4) gets skipped. **Single most-tested bug on AP CS A.**

ArrayList Traversal — enhanced for is read-only [3]

```
// Read-only — sum elements:
int sum = 0;
for (Integer x : list) sum += x;

// To MODIFY elements, use index-based:
for (int i = 0; i < list.size(); i++) {
    list.set(i, list.get(i) * 2);
}
```

PAGE 7 | UNIT 8: 2D ARRAYS — FRQ Q4 SUBJECT

2D Array Basics [3]/[4]

```
int[][] grid = {
    {1, 2, 3},      // row 0
    {4, 5, 6},      // row 1
    {7, 8, 9}       // row 2
};

grid.length        // 3    (number of ROWS)
grid[0].length      // 3    (number of COLUMNS in row 0)
grid[1][2]          // 6    (row 1, column 2)

// Initialize a 5x4 grid:
int[][] g = new int[5][4];    // 5 rows, 4 cols, all 0
```

Row-Major Traversal — the standard [3]/[4]

```
// ROW-MAJOR (most common): outer = rows, inner = columns
for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[0].length; c++) {
        System.out.print(grid[r][c] + " ");
    }
    System.out.println();
}

// COLUMN-MAJOR: outer = columns, inner = rows
for (int c = 0; c < grid[0].length; c++) {
    for (int r = 0; r < grid.length; r++) {
        // visits column c top-to-bottom, then column c+1, etc.
    }
}
```

Common 2D Algorithms [4]/[5]

```
// SUM all elements
int sum = 0;
for (int r = 0; r < g.length; r++) {
    for (int c = 0; c < g[0].length; c++) {
        sum += g[r][c];
    }
}

// SUM A SPECIFIC ROW r
int rowSum = 0;
for (int c = 0; c < g[r].length; c++) rowSum += g[r][c];

// SUM A SPECIFIC COLUMN c
int colSum = 0;
for (int r = 0; r < g.length; r++) colSum += g[r][c];

// FIND MAX (track row + col of max)
int max = g[0][0], maxR = 0, maxC = 0;
for (int r = 0; r < g.length; r++)
    for (int c = 0; c < g[0].length; c++)
        if (g[r][c] > max) { max = g[r][c]; maxR = r; maxC = c; }
```

KEY INSIGHT [4] Enhanced for on 2D: `for (int[] row : grid) { for (int x : row) { ... } }` — outer loops over rows (each row is `int[]`), inner over elements. Read-only.

PAGE 8 | UNITS 9-10: INHERITANCE · POLYMORPHISM · RECURSION

Inheritance — extends [3]/[4]

```

public class Animal {
    private String name;
    public Animal(String n) { name = n; }
    public String getName() { return name; }
    public String speak() { return "generic sound"; }
}

public class Dog extends Animal {
    private String breed;
    public Dog(String n, String b) {
        super(n);          // MUST call super constructor first
        breed = b;
    }
    @Override
    public String speak() { return "Woof!"; } // OVERRIDES parent
}

```

super Keyword [4]

- `super(args)` [4] = calls PARENT'S constructor. **Must be first line** of child constructor.
- `super.method()` [4] = calls PARENT'S version of an overridden method.
- **Inherited methods** [3]/[4]: child has all PUBLIC methods of parent. Private members are inherited but NOT directly accessible.

Polymorphism [4]/[5]

```

Animal[] pets = new Animal[3];
pets[0] = new Dog("Rex", "Lab");
pets[1] = new Cat("Whiskers");
pets[2] = new Animal("Generic");

for (Animal a : pets) {
    System.out.println(a.speak());
    // Calls Dog's speak, Cat's speak, Animal's speak -
    // Java picks at RUNTIME based on actual object type
}

```

KEY INSIGHT [5] **Dynamic dispatch.** `Animal a = new Dog(...);` — `a.speak()` runs **DOG's** `speak` (actual object type), even though declared as `Animal`. This is polymorphism's whole point.

Object class — equals + toString [4]/[5]

- **Every class extends Object** by default. `Object` provides default `equals()` (uses `==`) and `toString()` (returns `class@hexAddress`).
- [4] **Override** `toString()` so printing shows readable info.
- [5] **Override** `equals(Object other)` for content equality. Cast `other` to your class and compare fields.

Recursion [4]/[5]

```

// Every recursive method needs:
// 1. BASE CASE (stops the recursion)
// 2. RECURSIVE CALL (with smaller / different input)

public static int factorial(int n) {
    if (n <= 1) return 1;          // base case
    return n * factorial(n - 1);   // recursive call
}

public static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

// Reverse a String recursively
public static String reverse(String s) {
    if (s.length() <= 1) return s;
    return reverse(s.substring(1)) + s.charAt(0);
}

```

WATCH OUT [4] **No base case = infinite recursion = StackOverflowError.** On MCQ tracing, watch for missing base case → answer is "runtime error" or "stack overflow".

PAGE 9 | FRQ STRATEGY · 4-Q FORMULA · COMMON CODE TRAPS

THE 4 FRQs — predictable structure [3]/[4]

Q	Topic	Recipe
Q1	Methods + control flow	Write 1-2 methods using arrays/Strings + conditionals/loops. Lowest difficulty. Get all 9 points.
Q2	Class design	Write a complete class. Constructor + instance vars + 1-3 methods. Use the universal class template.
Q3	Array OR ArrayList	Method that traverses, manipulates, or transforms a 1-D collection. Watch the remove pitfall .
Q4	2D Array	Row-major or column-major traversal. Track row + col when finding extremes.

FRQ Code-Writing Discipline [3]/[4]/[5]

- [3] **Match method header EXACTLY** — return type, parameters, visibility (public). Wrong header = 0 on the method-header point.
- [3] **Use the methods they describe** — don't reimplement helper methods that already exist.
- [4] **Edge cases**: empty array, single-element, negatives, the boundary indices. Test mentally.
- [4] **Off-by-one**: loop bounds usually `i < arr.length` (NOT `<=`).
- [5] **Name your method's variables clearly** — graders read fast; obvious code earns the point.
- [5] **Don't write "pseudocode-y" Java**. Real Java with semicolons, braces, declared types. The exam books accept hand-written code; clarity matters more than perfection.

Common MCQ Traps — single-point losses [3]/[4]

- [3] **Integer division**: `5/2 = 2`, not 2.5.
- [3] **String == comparison**: never works for content. Use `.equals()`.
- [3] **Array length** is `arr.length` (no parens); String length is `s.length()` (with parens); ArrayList size is `list.size()`.
- [4] **ArrayList remove + i++** skips next element. Trace pace by pace.
- [4] **Missing super() call** in constructor → class won't compile or default `super()` runs (might fail if no no-arg parent constructor).
- [4] **Static methods** can't access instance vars without an object reference.
- [5] **Polymorphism**: declared type vs. actual type. Method picked by actual type at runtime, but declared type controls WHICH methods you can call.
- [5] **Recursion trace**: build a call stack on paper. Don't try in your head.

PAGE 10 | SCORE LADDER · TIER CHECKLISTS · THE ONE THING

THE SCORE LADDER

For a 3 (Pass)	For a 4	For a 5
MCQ: 22-26 / 40	MCQ: 28-32 / 40	MCQ: 33-38 / 40
FRQ Q1: solid (~7/9). Q2: write the class shell. Q3 + Q4: partial credit.	Q1 + Q2 nearly perfect. Q3 watch remove. Q4 standard 2D traversal.	All 4 FRQs near-perfect. Edge cases handled. Polymorphism + recursion clean.
Java basics: vars, if/else, for/while, arrays, Strings.	Add: ArrayList, classes, simple inheritance.	Add: polymorphism, recursion, 2D mastery, all algorithm patterns.
Avoid integer-division and == errors.	Override toString. Use super correctly.	Override equals. Trace recursion. Polymorphism in arrays of parent type.

TIER-SPECIFIC TEST-DAY CHECKLISTS

For a 3 — focus only on these:

- ✓ MCQ: 22+ correct.
- ✓ String comparison ALWAYS uses `.equals()`.
- ✓ Integer division traps: cast to double before dividing.
- ✓ Array traversal: `for (int i = 0; i < arr.length; i++)`.
- ✓ Q1 FRQ: get full credit by writing simple correct loops.

For a 4 — add these on top:

- ✓ MCQ: 28+ correct.
- ✓ Q2 FRQ: complete class with constructor + private fields + getters + 1-2 methods.
- ✓ ArrayList remove: count down OR don't increment after removing.
- ✓ 2D arrays: standard row-major nested loops. Track row + col.
- ✓ Inheritance: `super(args)` first, `@Override` annotation, `super.method()` for parent version.

For a 5 — add these on top:

- ✓ MCQ: 33+ correct.
- ✓ Q3 + Q4: full credit. Edge cases (empty array, 1 element, all-same).
- ✓ Polymorphism: array of parent type holding child objects, dynamic dispatch.
- ✓ Recursion trace: build call stack on paper. Identify base case + recursive case.
- ✓ Override `equals(Object other)` properly: cast + compare fields.
- ✓ Algorithms: trace insertion/selection/binary search.

THE ONE THING THAT MATTERS MOST [3][4][5]: WRITE EXACT JAVA, NOT PSEUDOCODE. AP CS A graders are looking for actual compilable Java. Semicolons after every statement. Curly braces around blocks. Declare types on every variable. Match the method header exactly. **Compilable code with one logic error scores far higher than pseudocode that "shows the right idea"**. When in doubt, write more not less — partial code that does most of the task earns most of the points.

If you do nothing else: master the universal class template, the array/2D traversal patterns, and the ArrayList remove pitfall. That's the AP CS A formula.

DISCLOSURE & TERMS OF USE

This study guide is published by **Wayfinder** (wayfinderai.org) for educational and self-study purposes only.

Not affiliated with the College Board

AP®, Advanced Placement®, and the names of all individual AP exams are registered trademarks of the College Board, which neither sponsors nor endorses this guide. Wayfinder is an independent educational publisher with no affiliation, partnership, or formal relationship with the College Board, ETS, or any other testing organization. Any references to the College Board, the AP Program, exam frameworks, scoring rubrics, or Course and Exam Descriptions are made for educational commentary, criticism, and study purposes consistent with fair use.

No guarantee of results

Wayfinder makes no representation, warranty, or guarantee — express or implied — that use of this guide will produce any particular AP exam score, college admission outcome, course grade, or academic result. AP exam performance depends on many factors outside our control, including the student's preparation history, individual aptitude, exam-day conditions, scoring rubric changes, and the work each student puts in. The strategies, frameworks, sample responses, and practice content in this guide reflect our best understanding of the relevant AP exam at time of publication; the College Board updates its exams and scoring guidelines periodically and may have done so since this guide was written.

Educational use only

The content in this guide is provided "as is" for the personal educational use of the original purchaser. It is not a substitute for the student's own classroom instruction, official College Board materials (the Course and Exam Description, released free-response questions, or AP Classroom), guidance from a qualified teacher, or independent verification of facts. We strongly encourage students to use this guide alongside — not in place of — official AP resources and their own school's curriculum.

Sample responses

Any sample student responses, model essays, exemplar paragraphs, or scored sample answers in this guide are illustrative compositions written by Wayfinder for instructional purposes. They are not actual student work, are not actual College Board materials, and have not been scored by official AP readers. They demonstrate writing patterns and rubric-aligned reasoning rather than guaranteed-score work.

Intellectual property

This guide is the copyrighted work of Wayfinder. You may print one copy for your personal study use. You may not redistribute, resell, repost, share via file-sharing services, upload to study-help platforms (e.g., Course Hero, Chegg, Quizlet), or use any portion to train AI/ML models without our express written permission. Quotations of brief passages for review or commentary purposes are permitted with attribution.

Limitation of liability

To the maximum extent permitted by law, Wayfinder, its affiliates, contributors, and licensors disclaim all liability for any direct, indirect, incidental, consequential, special, or exemplary damages arising from or related to the use of this guide, including but not limited to errors or omissions in content, reliance on study strategies, score outcomes, missed admission deadlines, or any other educational or financial result. This guide is provided without warranty of any kind, whether express or implied, including without limitation any implied warranty of merchantability, fitness for a particular purpose, or non-infringement.

Errors and updates

We aim for accuracy but errors are possible. If you find a mistake — a misstated fact, a stale rubric reference, a broken example — please email **support@wayfinderai.org** so we can correct it in the next revision. Use of this guide constitutes acknowledgment that exam content may have changed since publication and that you are responsible for cross-checking critical information against the most current official AP source.

Governing terms

Use of this guide is governed by Wayfinder's full Terms of Service and Privacy Policy, available at wayfinderai.org/terms.html and wayfinderai.org/privacy.html. By using this guide you agree to those terms.

© Wayfinder. All rights reserved.